Assignment 9: Feature Scaling and Model Evaluation

Q1: Feature Scaling Concepts

Explain the following:

- Standartization
- Min-Max Scaling
- Robust scalling
- Why feature scaling is important

Standartization: scales data so that mean = 0 and std = 1, with formula `z = (x-mean)/std`.

It is helpfull for most of the classic algorithms that uses normal distribution but for outliers

Min-Max Scaling: converts data into fixed range, formula `x_norm = (x - x_min) / (x_max - x_min)`

It is helpfull when we need to save our firstly distribution but if there is an outliers data could be....

Robust Scaling: Utilizes the median and the Interquartile Range (IQR) instead of the mean and standard deviation. Formula `x_robust = (x-median) / IQR`

Without scaling, a feature with a large range of values will dominate the other features. So Many machine learning algorithms perform significantly faster and more accurately on scaled data.

Q2: Load and Explore Dataset

In [54]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import seaborn as sns
from sklearn.datasets import load_breast_cancer
```

In [55]:
```
df2 = load_breast_cancer()
df = pd.DataFrame(df2.data, columns=df2.feature_names)
df.head()
```

Out[55]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	radius error	texture error	peri
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	1.0950	0.9053	
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	0.5435	0.7339	
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	0.7456	0.7869	
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	0.4956	1.1560	
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	0.7572	0.7813	

In [56]:
```
df["target"] = df2.target
df["target_name"] = df["target"].map({0: "malignant", 1: "benign"})
```

Q3: Train-Test Split

Split data into training and testing sets (80/20)

In [ ]:
```
from sklearn.model_selection import train_test_split

y = df2.target
X = df2.data

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
```

```
    stratify=y
)
```

Q4: Apply Feature Scaling

Apply:

- StandardScaler
- MinMaxScaler
- RobustScaler

In [58]:
```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
```

In [ ]:
```
scaler1 = StandardScaler()
X_train_std1 = scaler1.fit_transform(X_train)
X_test_std1 = scaler1.transform(X_test)
```

In [74]:
```
scaler2 = MinMaxScaler()
X_train_std2 = scaler2.fit_transform(X_train)
X_test_std2 = scaler2.transform(X_test)
```

In [72]:
```
scaler3 = RobustScaler()
X_train_std3 = scaler3.fit_transform(X_train)
X_test_std3 = scaler3.transform(X_test)
```

Q5: Train Models

Train Logistic Regression on:

1. Unscaled data
2. Standardized scaled data
3. MinMax scaled data
4. Robust Scaled data

In [62]:
```
from sklearn.linear_model import LogisticRegression
```

In [63]:
```
lr = LogisticRegression(max_iter=10000)
lr.fit(X_train, y_train)
```

Out[63]:
```
▾ LogisticRegression
▶ Parameters
```

In [ ]:
```
lr1 = LogisticRegression(max_iter=10000)
lr1.fit(X_train_std1, y_train)
```

Out[ ]:
```
▾ LogisticRegression
▶ Parameters
```

In [65]:
```
lr2 = LogisticRegression(max_iter=10000)
lr2.fit(X_train_std2, y_train)
```

Out[65]:
```
▾ LogisticRegression
▶ Parameters
```

In [66]:
```
lr3 = LogisticRegression(max_iter=10000)
lr3.fit(X_train_std3, y_train)
```

Out[66]:
```
▾ LogisticRegression
▶ Parameters
```

Q6: Evaluate Each Model

Evaluate each model using the corresponding test set and print the following:

- Accuracy
- MAE
- RMSE

In [67]:
```
from sklearn.metrics import accuracy_score as acc, mean_absolute_error as mae, root_mean_squared_error as rmse
```

In [ ]:
```
models = {
    "Unscaled": (lr, X_test),
    "STD Scaling": (lr1, X_test_std1),
    "MinMax Scaling": (lr2, X_test_std2),
    "Robust Scaling": (lr3, X_test_std3),
}

rows = []
for name, (model, X_te) in models.items():
    y_pred = model.predict(X_te)
    rows.append({
        "Scaler type": name,
        "Accuracy": acc(y_test, y_pred),
        "MAE": mae(y_test, y_pred),
        "RMSE": rmse(y_test, y_pred)
    })

results = pd.DataFrame(rows)
results
```

Out[ ]:

	Scaler type	Accuracy	MAE	RMSE
0	Unscaled	0.964912	0.035088	0.187317
1	STD Scaling	0.596491	0.403509	0.635223
2	MinMax Scaling	0.964912	0.035088	0.187317
3	Robust Scaling	0.982456	0.017544	0.132453

Applying feature scaling significantly improves the model's accuracy and reduces errors compared to using unscaled data. Additionally, all three scaling methods (Standardization, Min-Max, and Robust) yield very similar performance, showing that the act of scaling is more critical here than the specific technique chosen.

Q8: Reflection

Discuss:

- Impact of scaling on performance
- When scaling is necessary

1. Scaling allows gradient descent algorithms to converge faster (finding optimal weights more efficiently) and distance-based algorithms (KNN) to correctly calculate distances between data points. As a result, accuracy metrics and model stability often increase significantly compared to the same algorithm applied to raw (unscaled) data.

2. It is required for algorithms whose mechanics depend on the scale of the numbers (e.g., KNN, SVM, K-Means, Logistic Regression, Linear Regression, PCA, and Neural Networks).

Scaling is also mandatory when applying regularization techniques (L1/Lasso, L2/Ridge). Conversely, it is not required for tree-based algorithms (Decision Trees, Random Forest, XGBoost), as they make decisions based on feature thresholds and are independent of absolute magnitudes